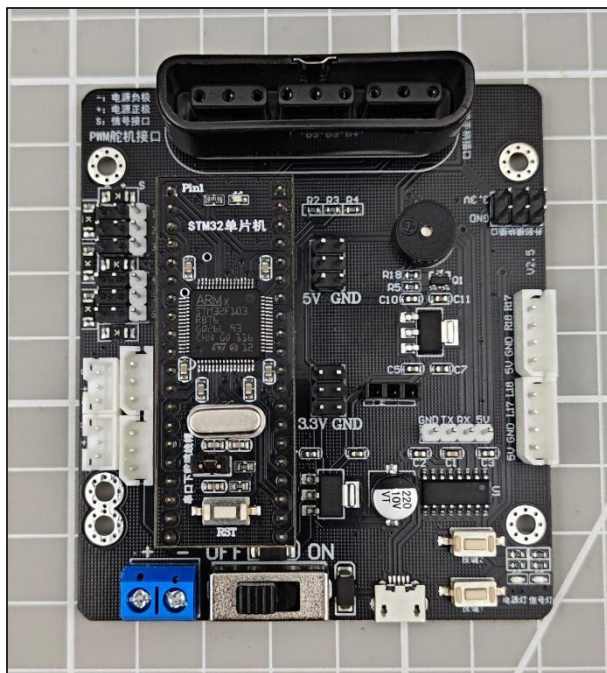


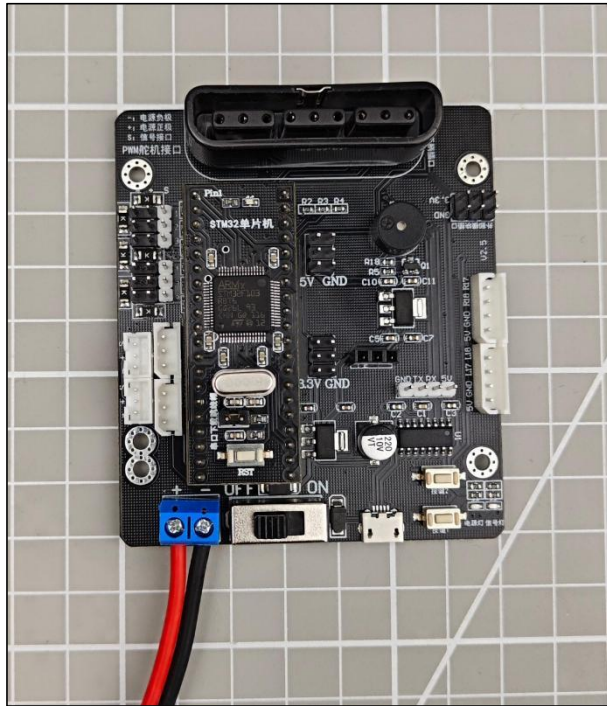
# STM32 版本开发教程

## 1. 准备工作

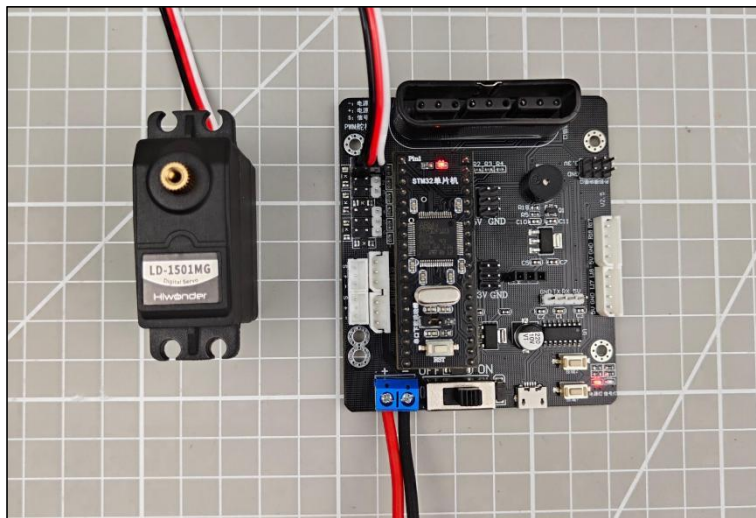
本节示例使用的是 STM32 开发板和开源 6 路舵机控制器，通过 7.4V 2200mAh 锂电池来供电。将 STM32 开发板安装至开源 6 路舵机控制器的单片机模块插座上。



- 1) 将电池对接线连接到舵机控制器的电源接口，并接上锂电池（**注意：红色接“+”，黑色接“-”，正负极不要接反**）。



2) 将舵机接入控制板的 1 号接口。



---

**注意：**

- ① 如使用我司锂电池，连接锂电池对接线请以红接“+”，黑接“-”接到 DC 接口。
  - ② 如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路而造成短路。
-

## 1.2 环境配置

在电脑端安装 Keil5，软件包位于“2.软件工具->STM32 软件工具”下。关于 Keil5 的详细使用，可在对应目录下进行学习。

## 2. 案例开发

开源 6 路舵机控制器 51 单片机主控的程序结构如下：

| 文件名称         | 程序内容                            |
|--------------|---------------------------------|
| Bluetooth    | 蓝牙通信程序库文件                       |
| Flash        | 将动作组保存/读取的底层程序库文件               |
| PS2GamePad   | PS2 手柄通信的底层程序库文件                |
| PCMsg        | 上位机通信的底层库文件                     |
| PWM          | PWM 舵机控制的底层库文件                  |
| RobotRun     | 动作组控制的底层库文件                     |
| BusServoCtrl | 总线舵机控制底层库文件                     |
| App          | 应用层文件，里面构造了上位机控制以及手柄控制的应用层函数接口。 |

1) 本程序主要应用层任务函数均在 App.c 文件中，因此我们仅针对 App.c 函数进行详细说明，其他文件可作为库文件使用，内容无需修改可直接调用。

2) 以下创建了一些我们在程序中需要使用的一些变量，具体含义如下：

```
10 u32 gSystemTickCount = 0; //系统从启动到现在的毫秒数
11
12 uint8 BuzzerState = 0;
13 uint16 Ps2TimeCount = 0;
14
15 uint16 BatteryVoltage;
```

**gSystemTickCount**: 用于存放系统从启动到当前运行代码的运行时间, 单位为 ms。

**BuzzerState**: 控制蜂鸣器的开关变量, 当它为 1 时, 表示允许蜂鸣器发声。

**Ps2TimeCount**: 用于设定按键检测 1 次的间隔时间。

**BatteryVoltage**: 存放当前电压值。

3) 下面是延时函数, DelayMs 是毫秒级延时, DelayUs 是微妙级延时。

```
29 //延时rms
30 //注意rms的范围
31 //SysTick->LOAD为24位寄存器, 所以, 最大延时为:
32
33 //rms<=0xfffff*8*1000/SYSCLK
34 //SYSCLK单位为Hz, rms单位为ms
35 //对72M条件下, rms<=1864
36 void DelayMs(u16 rms)
37 {
38     u32 temp;
39     SysTick->LOAD=(u32)rms*fac_ms; //时间加载(SysTick->LOAD为24bit)
40     SysTick->VAL =0x00;           //清空计数器
41     SysTick->CTRL=0x01;           //开始倒数
42     do
43     {
44         temp=SysTick->CTRL;
45     }
46     while(temp&0x01&&!(temp&(1<<16))); //等待时间到达
47     SysTick->CTRL=0x00;           //关闭计数器
48     SysTick->VAL =0x00;           //清空计数器
49 }
50 //延时nus
51 //nus为要延时的us数.
52 void DelayUs(u32 nus)
53 {
54     u32 temp;
55     SysTick->LOAD=nus*fac_us; //时间加载
56     SysTick->VAL=0x00;         //清空计数器
57     SysTick->CTRL=0x01;         //开始倒数
58     do
59     {
60         temp=SysTick->CTRL;
61     }
62     while(temp&0x01&&!(temp&(1<<16))); //等待时间到达
63     SysTick->CTRL=0x00;         //关闭计数器
64     SysTick->VAL =0x00;         //清空计数器
65 }
```

4) 下面是初始化 LED 和按键 1 的引脚为输出模式。



```

71 void InitLED(void)
72 {
73
74     GPIO_InitTypeDef GPIO_InitStructure;
75
76     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE); //使能PA端口时钟
77     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9; //LED0-->PC.2 端口配置
78     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
79     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
80     GPIO_Init(GPIOB, &GPIO_InitStructure);
81 }
82
83
84 void InitKey(void)
85 {
86
87     GPIO_InitTypeDef GPIO_InitStructure;
88     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);
89     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
90     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
91     GPIO_Init(GPIOC, &GPIO_InitStructure);
92 }
93

```

##### 5) 初始化蜂鸣器为推挽输出模式。

```

94 void InitBuzzer(void)
95 {
96
97     GPIO_InitTypeDef GPIO_InitStructure;
98
99     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE); //使能端口时钟
100
101     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13;
102     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
103     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
104     GPIO_Init(GPIOC, &GPIO_InitStructure);
105 }
106

```

##### 6) 初始化定时器 2.

```

108 void InitTimer2(void) //100us
109 {
110     NVIC_InitTypeDef NVIC_InitStructure;
111
112     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
113     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE); //时钟使能
114
115     TIM_TimeBaseStructure.TIM_Period = (10 - 1); //设置在下一个更新事件装入活动的自动重装载寄存
116     TIM_TimeBaseStructure.TIM_Prescaler = (720-1); //设置用来作为TIMx时钟频率除数的预分频值
117     TIM_TimeBaseStructure.TIM_ClockDivision = 0; //设置时钟分割:TDTS = Tck_tim
118     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //TIM向上计数模式
119     TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure); //根据TIM_TimeBaseInitStruct中指定的参数初
120
121     TIM_ITConfig( //使能或者失能指定的TIM中断
122         TIM2, //TIM2
123         TIM_IT_Update | //TIM 中断源
124         TIM_IT_Trigger, //TIM 触发中断源
125         ENABLE //使能
126     );
127
128     TIM_Cmd(TIM2, ENABLE); //使能TIMx外设
129
130     NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn; //TIM2中断
131     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2; //先占优先级0级
132     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3; //从优先级3级
133     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ通道被使能
134     NVIC_Init(&NVIC_InitStructure); //根据NVIC_InitStruct中指定的参数初始化外设NVIC寄存器
135 }

```

## 7) 初始化 ADC 转换器, 启动 ADC1, 以单通道工作模式运行。

```
137 void InitADC(void)
138 {
139     ADC_InitTypeDef ADC_InitStructure;
140     GPIO_InitTypeDef GPIO_InitStructure;
141     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC | RCC_APB2Periph_ADC1, ENABLE); //使能ADC1通道时钟
142
143     RCC_ADCCLKConfig(RCC_PCLK2_Div6); //72M/6=12, ADC最大时间不能超过14M
144     //PA0/1/2/3 作为模拟通道输入引脚
145     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3; //GPIO_Pin_1|GPIO_Pin_2|GPIO_Pin_3;
146     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN; //模拟输入引脚
147     GPIO_Init(GPIOC, &GPIO_InitStructure);
148
149     ADC_DeInit(ADC1); //将外设 ADC1 的全部寄存器重设为缺省值
150
151     ADC_InitStructure.ADC_Mode = ADC_Mode_Independent; //ADC工作模式:ADC1和ADC2工作在独立模式
152     ADC_InitStructure.ADC_ScanConvMode = DISABLE; //模数转换工作在单通道模式
153     ADC_InitStructure.ADC_ContinuousConvMode = DISABLE; //模数转换工作在单次转换模式
154     ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; //转换由软件而不是外部触发启动
155     ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right; //ADC数据右对齐
156     ADC_InitStructure.ADC_NbrOfChannel = 1; //顺序进行规则转换的ADC通道的数目
157     ADC_Init(ADC1, &ADC_InitStructure); //根据ADC_InitStructure中指定的参数初始化外设ADCx的寄存器
158
159
160
161     ADC_Cmd(ADC1, ENABLE); //使能指定的ADC1
162
163     ADC_ResetCalibration(ADC1); //重置指定的ADC1的校准寄存器
164
165     while(ADC_GetResetCalibrationStatus(ADC1)); //获取ADC1重置校准寄存器的状态, 设置状态则等待
166
167     ADC_StartCalibration(ADC1); //开始指定ADC1的校准状态
168
169     while(ADC_GetCalibrationStatus(ADC1)); //获取指定ADC1的校准程序, 设置状态则等待
170
171     ADC_SoftwareStartConvCmd(ADC1, ENABLE); //使能指定的ADC1的软件转换启动功能
172 }
```

8) 获取电池电压的模拟值。其中  $ADC\_CONTR = ADC\_POWER \mid ADC\_SPEEDLL \mid ch \mid$ 

$ADC\_START$  控制转换器低速模式运行, 并从指定的  $ch$  引脚读取模拟值。

```
175 uint16 GetADCResult(BYTE ch)
176 {
177     //设置指定ADC的规则组通道, 设置它们的转化顺序和采样时间
178     ADC_RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5); //ADC1, ADC通道3, 规则采样顺序
179
180     ADC_SoftwareStartConvCmd(ADC1, ENABLE); //使能指定的ADC1的软件转换启动功能
181
182     while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); //等待转换结束
183
184     return ADC_GetConversionValue(ADC1); //返回最近一次ADC1规则组的转换结果
185 }
186
187
```

将读取到模拟传感器的值通过下面的函数对输入的电压模拟值进行处理, 最终得到当前电压的值, 单位为毫伏。

```
188 void CheckBatteryVoltage(void)
189 {
190     uint8 i;
191     uint32 v = 0;
192     for(i = 0; i < 8; i++)
193     {
194         v += GetADCResult(ADC_BAT);
195     }
196     v >>= 3;
197
198     v = v * 2475 / 1024; //adc / 4096 * 3300 * 3 (3代表放大3倍, 因为采集电压时电阻分压了)
199     BatteryVoltage = v;
200
201 }
202
```

最后，构造一个输出电源电压值的函数：GetBatteryVoltage()。

```
203 uint16 GetBatteryVoltage(void)
204 { //电压毫伏
205     return BatteryVoltage;
206 }
207
```

9) 这里初始化了定时器 2，使其每隔 100us 产生 1 次中断，主要用于蜂鸣器控制。

```
108 void InitTimer2(void) //100us
109 {
110     NVIC_InitTypeDef NVIC_InitStructure;
111
112     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
113     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE); //时钟使能
114
115     TIM_TimeBaseStructure.TIM_Period = (10 - 1); //设置在下一个更新事件装入活动的自动重装载寄存器周期的值
116     TIM_TimeBaseStructure.TIM_Prescaler = (720-1); //设置用来作为TIMx时钟频率除数的预分频值
117     TIM_TimeBaseStructure.TIM_ClockDivision = 0; //设置时钟分割:TDTS = Tck_tim
118     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //TIM向上计数模式
119     TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure); //根据TIM_TimeBaseInitStruct中指定的参数初始化TIMx的时间基数单位
120
121     TIM_ITConfig( //使能或者失能指定的TIM中断
122         TIM2, //TIM2
123         TIM_IT_Update | //TIM 中断源
124         TIM_IT_Trigger, //TIM 触发中断源
125         ENABLE //使能
126     );
127
128     TIM_Cmd(TIM2, ENABLE); //使能TIMx外设
129
130     NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn; //TIM2中断
131     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2; //先占优先级0级
132     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3; //从优先级3级
133     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ通道被使能
134     NVIC_Init(&NVIC_InitStructure); //根据NVIC_InitStruct中指定的参数初始化外设NVIC寄存器
135 }
```

10) 这里是一个控制蜂鸣器鸣响的函数。

```
208 void Buzzer(void)
209 { //放到100us的定时中断里面
210     static bool fBuzzer = FALSE;
211     static uint32 t1 = 0;
212     static uint32 t2 = 0;
213     if(fBuzzer)
214     {
215         if(++t1 >= 2)
216         {
217             t1 = 0;
218             BUZZER = !BUZZER; //2.5KHz
219         }
220     }
221     else
222     {
223         BUZZER = 0;
224     }
225 }
```

其中，fBuzzer 变量为控制蜂鸣器是否发出声音的一个标志变量，当值为 FALSE 时蜂鸣器停止发声，当值为 TRUE 时蜂鸣器发声。我们通过将 BUZZER 引脚置位或复位的方式控制蜂鸣器发出声音，若 BUZZER 为高电平则蜂鸣器发出声音，若为低电平则停止发声。t1 变量来控制单次发声的时长，t2 变量来控制每次发声之间的时间间隔。



```
227     if(BuzzerState == 0)
228     {
229         fBuzzer = FALSE;
230         t2 = 0;
231     }
232     else if(BuzzerState == 1)
233     {
234         t2++;
235         if(t2 < 5000)
236         {
237             fBuzzer = TRUE;
238         }
239         else if(t2 < 10000)
240         {
241             fBuzzer = FALSE;
242         }
243         else
244         {
245             t2 = 0;
246         }
247     }
248 }
```

11) 通过定时器 2 产生中断，在中断短期间调用 Buzzer() 函数控制蜂鸣器是否发声。

```
250 BOOL manual = FALSE;
251 void TIM2_IRQHandler(void)    //TIM2中断
252 { //定时器2中断 100us
253     static uint32 time = 0;
254     static uint16 timeBattery = 0;
255     if(TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET) //检查指定的TIM中断发生与否:TIM 中断源
256     {
257         TIM_ClearITPendingBit(TIM2, TIM_IT_Update); //清除TIMx的中断待处理位:TIM 中断源
258
259         Buzzer();
260         if(++time >= 10)
261         {
262             time = 0;
263             gSystemTickCount++;
264             Ps2TimeCount++;
265             if(GetBatteryVoltage() < 6400) //小于6.4V报警
266             {
267                 timeBattery++;
268                 if(timeBattery > 5000) //持续5秒
269                 {
270                     BuzzerState = 1;
271                 }
272             }
273             else
274             {
275                 timeBattery = 0;
276                 if(manual == FALSE)
277                 {
278                     BuzzerState = 0;
279                 }
280             }
281         }
282     }
283 }
284 }
```

当定时器 2 中断时，中断服务程序会被调用，进入中断服务程序后我们创建了 time、timeBattery 两个变量用于跟踪时间和状态。当 time 值为 10 时（即定时 1ms），我们将 gSystemTickCount 变量加 1，表示程序从启动到此时经历的时间增加了 1ms。然后再判断当前的电源电压，当电压小于 6.4V，将 timeBattery 变量加 1，用于计算电压保持在 6.6V 以下的持续时间，当持续时间长达 5 秒，将 BuzzerState 状态置 1，允许蜂鸣器发声。



12) 以下为更新舵机转动角度的函数，每 20ms 更新一次舵机状态，关于 ServoPwmDutyCompare() 函数的含义我们在后面的舵机控制程序文件中介绍。

```
286 void TaskTimeHandle(void)
287 {
288     static uint32 time = 10;
289     static uint32 times = 0;
290     if(gSystemTickCount > time)
291     {
292         time += 10;
293         times++;
294         if(times % 2 == 0)//20ms
295         {
296             ServoPwmDutyCompare();
297         }
298         if(times % 50 == 0)//500ms
299         {
300             CheckBatteryVoltage();
301         }
302     }
303 }
304
305
```

13) 以下为舵机控制的任务接口函数。

```
306 int16 BusServoPwmDutySet[8] = {500, 500, 500, 500, 500, 500, 500, 500};
307 uint8 i;
308 void TaskRun(void)
309 {
310     static bool Ps2State = FALSE;
311     static uint8 mode = 0;
312     uint8 PS2KeyValue;
313     static uint8 keycount = 0;
314     TaskTimeHandle();
315
316     TaskPCMsgHandle();
317     TaskBLEMsgHandle();
318     TaskRobotRun();
319
320     if(KEY == 0)
321     {
322         DelayMs(60);
323         {
324             if(KEY == 0)
325             {
326                 keycount++;
327             }
328             else
329             {
330                 if (keycount > 20)
331                 {
332                     keycount = 0;
333                     FullActRun(100, 0);
334                     return;
335                 }
336                 else
337                 {
338                     keycount = 0;
339                     LED = ~LED;
340                     FullActRun(100, 1);
341                 }
342             }
343         }
344     }
345 }
```

在这里创建了 Ps2State（用于标志 PS2 手柄是否连接）、mode（控制模式）、PS2KeyValue（记录 PS2 手柄按下的按键）三个变量，keycount 记录按键按下的次数。随后调用到舵机动作更新的函数、电源电压检测函数、上位机控制函数、蓝牙控制函数以及动作组运行控制函数。

随后检测控制器板载的按键 1 是否按下，当 KEY 等于 0 表示按键按下，此时会将 keycount 变量加 1，当 keycount 计数大于 20 时，我们认为按键此时被长按，则通过 FullActRun 函数调用 100 号动作组无限运行。如果按键短按，那么只运行 1 次。

```
347 if(Ps2TimeCount > 50)
348 {
349     Ps2TimeCount = 0;
350     PS2KeyValue = PS2_DataKey();
351     if(mode == 1)
352     {
353         if( PS2_Button( PSB_SELECT ) & PS2_ButtonPressed( PSB_START ) )
354         {
355             mode = 0;
356             Ps2State = 0;
357             manual = TRUE;
358             BuzzerState = 1;
359             LED=~LED;
360             DelayMs(80);
361             manual = FALSE;
362             DelayMs(50);
363             manual = TRUE;
364             BuzzerState = 1;
365             DelayMs(80);
366             manual = FALSE;
367             LED=~LED;
368         }
369     }
```

此外我们通过 Ps2TimeCount 变量来作为检测一次手柄按键的时间间隔，当变量值大于 50ms 时，PS2\_DataKey 函数获取到按键的信息，当 Mode 变量为 1 时，表示当前为控制模式 1（即单舵机控制模式），在该模式下首先检测 select 和 start 按键是否同时按下，若同时按下则切换模式，我们在将 Mode 变量设置为 0（即动作组控制模式）。

manual 变量设置为 TRUE 可以控制蜂鸣器发生，LED 输出信号取反可以使 LED2 信号灯闪烁。

```
369 else
370 {
371     if(PS2KeyValue && !PS2_Button(PSB_SELECT))
372     {
373         LED=~LED;
374     }
375 }
376 switch( PS2KeyValue )
377 {
378     //根据按下的按键，控制舵机转动
379     case PSB_PAD_LEFT:
380         ServoSetPluseAndTime( 6, ServoPwmDutySet[6] + 20, 50 );
381         BusServoPwmDutySet[6] = BusServoPwmDutySet[6] + 10;
382         if (BusServoPwmDutySet[6] > 1000)
383             BusServoPwmDutySet[6] = 1000;
384         BusServoCtrl(6, SERVO_MOVE_TIME_WRITE, BusServoPwmDutySet[6], 50);
385         break;
386     case PSB_PAD_RIGHT:
387         ServoSetPluseAndTime( 6, ServoPwmDutySet[6] - 20, 50 );
388         BusServoPwmDutySet[6] = BusServoPwmDutySet[6] - 10;
389         if (BusServoPwmDutySet[6] < 0)
390             BusServoPwmDutySet[6] = 0;
391         BusServoCtrl(6, SERVO_MOVE_TIME_WRITE, BusServoPwmDutySet[6], 50);
392         break;
393     case PSB_PAD_UP:
394         ServoSetPluseAndTime( 5, ServoPwmDutySet[5] - 20, 50 );
395         BusServoPwmDutySet[5] = BusServoPwmDutySet[5] - 10;
396         if (BusServoPwmDutySet[5] < 0)
397             BusServoPwmDutySet[5] = 0;
398         BusServoCtrl(5, SERVO_MOVE_TIME_WRITE, BusServoPwmDutySet[5], 50);
399         break;
```

如果 select 和 start 按键没有被同时按下，则根据当前手柄按下的按键，控制信号灯

闪烁 1 次，再控制指定舵机转动。如：上图中，左侧向左的按键被按下后，6 舵机向 2500 位置转动，每次转动的变化量为 20。剩余的实现方法类似，这里就不再一一介绍了。

```
544     else
545     {
546         if( PS2_Button( PSB_SELECT ) && PS2_ButtonPressed( PSB_START ) ) //检查是不是 SELECT按钮被按住，然后按下
547         {
548             mode = 1; //将模式变为1，就单舵机模式
549             FullActStop(); //停止动作组运行
550             Ps2State = 0; //清除动作组模式用到的标志。
551             ServoSetPluseAndTime( 1, 1500, 1000 ); //将机械臂的舵机都转到1500的位置
552             ServoSetPluseAndTime( 2, 1500, 1000 );
553             ServoSetPluseAndTime( 3, 1500, 1000 );
554             ServoSetPluseAndTime( 4, 1500, 1000 );
555             ServoSetPluseAndTime( 5, 1500, 1000 );
556             ServoSetPluseAndTime( 6, 1500, 1000 );
557             manual = TRUE;
558             BuzzerState = 1;
559             LED=~LED;
560             DelayMs(80);
561             manual = FALSE;
562             DelayMs(50);
563             manual = TRUE;
564             BuzzerState = 1;
565             DelayMs(80);
566             manual = FALSE;
567             LED=~LED;
568         }
```

若 Mode 变量为 0，表示当前为控制模式 1（即动作组控制模式），在该模式下首先检测 select 和 start 按键是否同时按下，若同时按下则切换模式，我们在将 Mode 变量设置为 1（即单舵机控制模式）。然后控制舵机转动到 1500 的位置（即中位），manual 变量设置为 TRUE 可以控制蜂鸣器发声，设置为 FALSE 可以使蜂鸣器停止发声。通过 DelayMs() 函数能够实现蜂鸣器滴滴两声。

```
571     if(PS2KeyValue && !Ps2State && !PS2_Button(PSB_SELECT))
572     {
573         LED=~LED;
574     }
575
576     switch(PS2KeyValue)
577     {
578         case 0:
579             if(Ps2State)
580             {
581                 Ps2State = FALSE;
582             }
583             break;
584
585         case PSB_START:
586             if(!Ps2State)
587             {
588                 FullActRun(0, 1);
589             }
590             Ps2State = TRUE;
591             break;
592
593         case PSB_PAD_UP:
594             if(!Ps2State)
595             {
596                 FullActRun(1, 1);
597             }
598             Ps2State = TRUE;
599             break;
```



如果 select 和 start 按键没有被同时按下，则控制信号灯闪烁 1 次，然后判断当前手柄按下的按键，当检测到按下的是 start 按键时，调用 0 号动作组 1 次，如果为左侧向上的按键，则调用 1 号动作组运行 1 次。上图为部分代码，实现方法都是类似的，我们就不一一介绍了。